Real-time Swarm Simulation

A Comprehensive Academic and Technical Report

Project Component for

Parallel and Distributing Computing (UCS645)

Submitted By:

Sujal Mallick	102303266
Vansh Bhasin	102303270
Ritwik Gupta	102303096
Abhishek Jambla	102303900

Under the guidance of

Dr. Saif Nalband
(Assistant Professor, DCSE)

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
PATIALA - 147004

MAY, 2026

Abstract

This comprehensive academic report details the design, theoretical foundations, implementation architecture, and performance analysis of a high-performance, real-time simulation of emergent flocking behavior (commonly known as "Boids"). Utilizing the NVIDIA CUDA parallel computing platform alongside the OpenGL rendering API, this project successfully models the complex emergent dynamics of up to 200,000 independent agents at interactive frame rates. The report provides an exhaustive exploration of the mathematical formalizations governing agent behavior, including separation, alignment, cohesion, predator-prey dynamics, and complex obstacle avoidance. Furthermore, it deeply analyzes the transition from a naive $O(N^2)$ algorithmic complexity to a highly scalable $O(N \cdot K)$ complexity via grid-based spatial hashing algorithms sorted by Thrust radix sorts. The graphics rendering pipeline is also thoroughly dissected, highlighting the utilization of zero-copy CUDA-OpenGL interoperability, dynamic level-of-detail (LOD) scaling, and instanced rendering to overcome massive geometry throughput bottlenecks. Finally, extensive performance benchmarking, a discussion of technical challenges (such as massive fragment overdraw), and potential future expansions into multi-GPU and machine learning integrations are presented.

Contents

Chapter 1

Introduction

1.1 Background and Motivation

The natural world provides countless examples of collective behavior where complex, globally coordinated patterns emerge from simple, localized interactions. From the murmuration of starlings to the schooling of fish and the swarming of insects, these biological phenomena exhibit highly synchronized movements without any centralized control structure. Understanding and replicating these emergent behaviors has significant implications not only in biological modeling and computer graphics but also in robotics, traffic optimization, and distributed systems.

In 1987, Craig Reynolds revolutionized the computer graphics industry by introducing the "Boids" algorithmic model. Reynolds demonstrated that the intricate flocking behavior of birds could be remarkably simulated using three rudimentary rules: collision avoidance (separation), velocity matching (alignment), and flock centering (cohesion). Each "boid" (bird-oid object) operates as an independent agent, observing only its immediate neighborhood and adjusting its trajectory accordingly.

1.2 The Computational Challenge

While the conceptual framework of the Boids algorithm is elegant and computationally inexpensive for a small number of agents, it suffers from a fundamental scalability problem. In a naive implementation, every agent must evaluate its distance relative to every other agent in the simulation space to determine which neighbors fall within its perception radius. For a simulation consisting of N agents, this results in an $O(N^2)$ time complexity per frame.

For real-time applications targeting 60 frames per second (FPS), modern Central Processing Units (CPUs) quickly hit an execution bottleneck. Even with mul-

tithreading across high-end consumer CPUs, simulating more than a few thousand agents becomes unfeasible due to memory bandwidth limitations and the sequential nature of traditional processors.

1.3 GPU Acceleration as a Paradigm Shift

To overcome this $O(N^2)$ limitation, this project introduces a massive parallelization strategy using General-Purpose computing on Graphics Processing Units (GPGPU) via the NVIDIA CUDA architecture. GPUs are fundamentally designed for high-throughput, data-parallel workloads. By mapping each individual agent to a distinct CUDA thread, the entire swarm's state can be updated concurrently.

However, raw compute power alone does not resolve the $O(N^2)$ complexity. To fully realize the potential of GPU acceleration, this project implements a specialized spatial hashing data structure. By partitioning the simulation space into a uniform grid and sorting agents into memory contiguous blocks using the Thrust library, the algorithm's complexity is reduced to $O(N \cdot K)$, where K is the average local density of neighbors.

1.4 Scope and Objectives

This report provides a detailed dissection of the Swarm Simulator project. The primary objectives of this research and implementation are:

- To formalize the mathematical models dictating agent steering behaviors, extending the base Boids model with predator-prey dynamics and environmental obstacles.
- To design and implement a highly optimized CUDA kernel architecture capable of utilizing shared memory, minimizing warp divergence, and maximizing Streaming Multiprocessor (SM) occupancy.
- To architect a robust spatial hashing system entirely on the GPU to bypass the $O(N^2)$ bottleneck.
- To implement a high-efficiency rendering pipeline utilizing CUDA-OpenGL interoperability, bypassing the PCI-Express bottleneck entirely through zero-copy device memory sharing.
- To benchmark and evaluate the performance limits, identifying hardware bottlenecks and algorithmic thresholds.

Chapter 2

Literature Review

2.1 Evolution of Swarm Intelligence Models

The simulation of flocking behavior began strictly within the domain of biological research, attempting to understand the evolutionary advantages of schooling and herding. Reynolds' (1987) formalization bridged the gap into computer science, providing a discrete, frame-by-frame computable model. Following Reynolds, various extensions were proposed, including Tu and Terzopoulos' (1994) artificial fishes, which introduced biomechanical modeling and sophisticated cognitive layers over the basic steering rules.

2.2 Parallelization of Agent-Based Models

As the desire for larger, more realistic simulations grew, particularly in the film and video game industries, researchers began exploring parallel architectures. Initial efforts focused on multi-core CPU threading using APIs like OpenMP and MPI (Message Passing Interface) for distributed clusters. However, inter-process communication latency heavily restricted real-time performance.

The advent of programmable shaders (CUDA and OpenCL) in the late 2000s catalyzed a paradigm shift. Early GPU flocking implementations relied on fragment shaders and ping-pong textures to store agent positions and velocities. With the introduction of CUDA, generalized memory pointers allowed for much more complex data structures, paving the way for the implementation of advanced spatial partitioning algorithms directly on the GPU.

2.3 Spatial Partitioning Data Structures

The core to optimizing any N-body problem is spatial partitioning. Traditional tree-based structures like Quadtrees, Octrees, and Kd-trees offer excellent $O(\log N)$ query times but are notoriously difficult to construct and balance efficiently in a highly parallel GPU environment.

Alternatively, uniform grid spatial hashing has emerged as the standard for massive real-time particle simulations. First proposed for GPU particle systems by Green (2008), spatial hashing transforms spatial coordinates into a 1D hash value, sorts the particles by this hash to ensure memory contiguity, and uses boundary arrays to permit $O(1)$ cell lookups. This project builds heavily upon this specific architecture due to its synergy with the CUDA memory coalescing model.

Chapter 3

Theoretical Framework and Mathematical Models

3.1 The Core Boids Algorithm

The simulation space is defined as a continuous 2D plane normalized between $[-1, 1]$. Each agent i is represented by a set of physical properties: a position vector $\mathbf{p}_i = (x_i, y_i)$, a velocity vector $\mathbf{v}_i = (vx_i, vy_i)$, and an instantaneous acceleration vector $\mathbf{a}_i$.

The instantaneous acceleration $\mathbf{a}_i$ is computed as the weighted summation of individual steering forces $\mathbf{F}$.

$$\mathbf{a}_i = w_{sep}\mathbf{F}_{sep} + w_{ali}\mathbf{F}_{ali} + w_{coh}\mathbf{F}_{coh} + \mathbf{F}_{env} \tag{3.1}$$

3.1.1 Separation Force

The separation rule enforces collision avoidance. An agent computes a vector pointing away from every neighbor within a strict separation radius. To ensure smoother behavior, the magnitude of the repulsive force is inversely proportional to the distance squared.

$$\mathbf{F}_{sep} = \sum_{j \neq i, |\mathbf{p}_i - \mathbf{p}_j| < R_{per}} \frac{\mathbf{p}_i - \mathbf{p}_j}{|\mathbf{p}_i - \mathbf{p}_j|^2} \tag{3.2}$$

Where R_{per} is the perception radius.

3.1.2 Alignment Force

The alignment rule encourages agents to travel in the same general direction as their flockmates. It computes the average velocity of all neighbors within the perception

radius and creates a steering force towards that heading.

$$\mathbf{F}_{ali} = \left(\frac{1}{K} \sum_{j \neq i, |\mathbf{p}_i - \mathbf{p}_j| < R_{per}} \mathbf{v}_j \right) \tag{3.3}$$

Where K is the total number of valid neighbors.

3.1.3 Cohesion Force

The cohesion rule ensures the flock stays together rather than dispersing. It determines the center of mass (average position) of the local neighborhood and applies a steering force towards it.

$$\mathbf{F}_{coh} = \left(\frac{1}{K} \sum_{j \neq i, |\mathbf{p}_i - \mathbf{p}_j| < R_{per}} \mathbf{p}_j \right) - \mathbf{p}_i \tag{3.4}$$

3.2 Advanced Dynamics and Environmental Integration

3.2.1 Predator-Prey Interactions

To simulate complex biological ecosystems, the system supports heterogeneous agent types defined via an `AgentType` enumeration: `PREY` and `PREDATOR`.

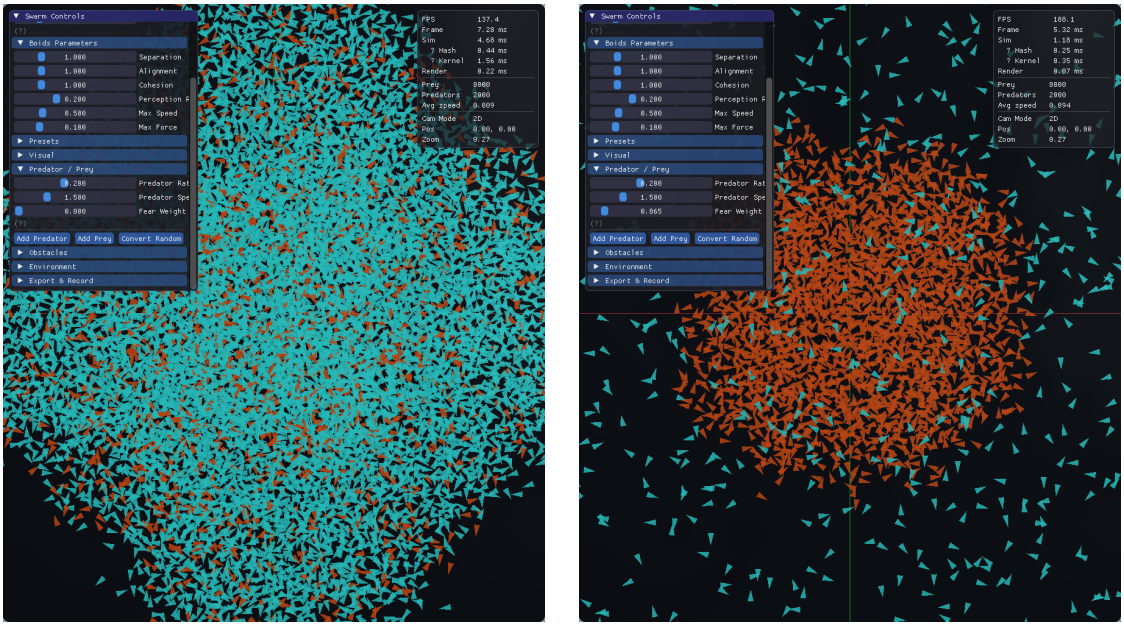
When an agent of type `PREDATOR` evaluates its neighbors, it actively seeks the nearest `PREY` agent and applies a high-magnitude pursuit vector. Conversely, when a `PREY` agent detects a `PREDATOR` within its perception radius, it introduces a "Fear Weight" parameter. This artificially amplifies the magnitude of the separation force strictly relative to the predator, overriding alignment and cohesion rules and inducing emergent scattering behavior.

3.2.2 GPU-Accelerated Obstacle Avoidance

To create dynamic environments, a ray-casting collision avoidance system runs within the GPU kernel. Each agent projects a look-ahead vector based on its current velocity:

$$\mathbf{p}_{ahead} = \mathbf{p}_i + \hat{\mathbf{v}}_i \cdot D_{lookahead} \tag{3.5}$$

The system evaluates $\mathbf{p}_{ahead}$ against an array of geometric obstacles (Circles, Axis-Aligned Bounding Boxes, Line Segments) residing in GPU memory.



Zero Fear Weight High Fear Weight

Figure 3.1: Visualizing the emergent predator-prey dynamics. A high Fear Weight parameter forces the prey to prioritize extreme separation, resulting in rapid radial scattering.

For circular obstacles with center $\mathbf{c}$ and radius r: If $|\mathbf{p}_{ahead} - \mathbf{c}| < r + D_{safety}$, an avoidance steering vector is generated perpendicular to the obstacle surface:

$$\mathbf{F}_{avoid} = \frac{\mathbf{p}_{ahead} - \mathbf{c}}{|\mathbf{p}_{ahead} - \mathbf{c}|} \cdot w_{avoid} \tag{3.6}$$

3.2.3 Further Environmental Factors and Kinematics

To create a highly interactive and varied simulation, several other parameters mathematically manipulate the swarm's trajectory outside of the core Boids rules:

- **Global Wind (W):** A constant directional force uniformly applied to the acceleration vector of all agents across the simulation grid, simulating environmental currents.
- **Attractors and Repulsors:** Localized points of interest (often tied to the user's mouse position). If an agent is within the attractor radius R_{attr}, a force is applied inversely proportional to its distance from the center. Repulsors act identically but with a negative weight, scattering agents that come too close to the cursor.

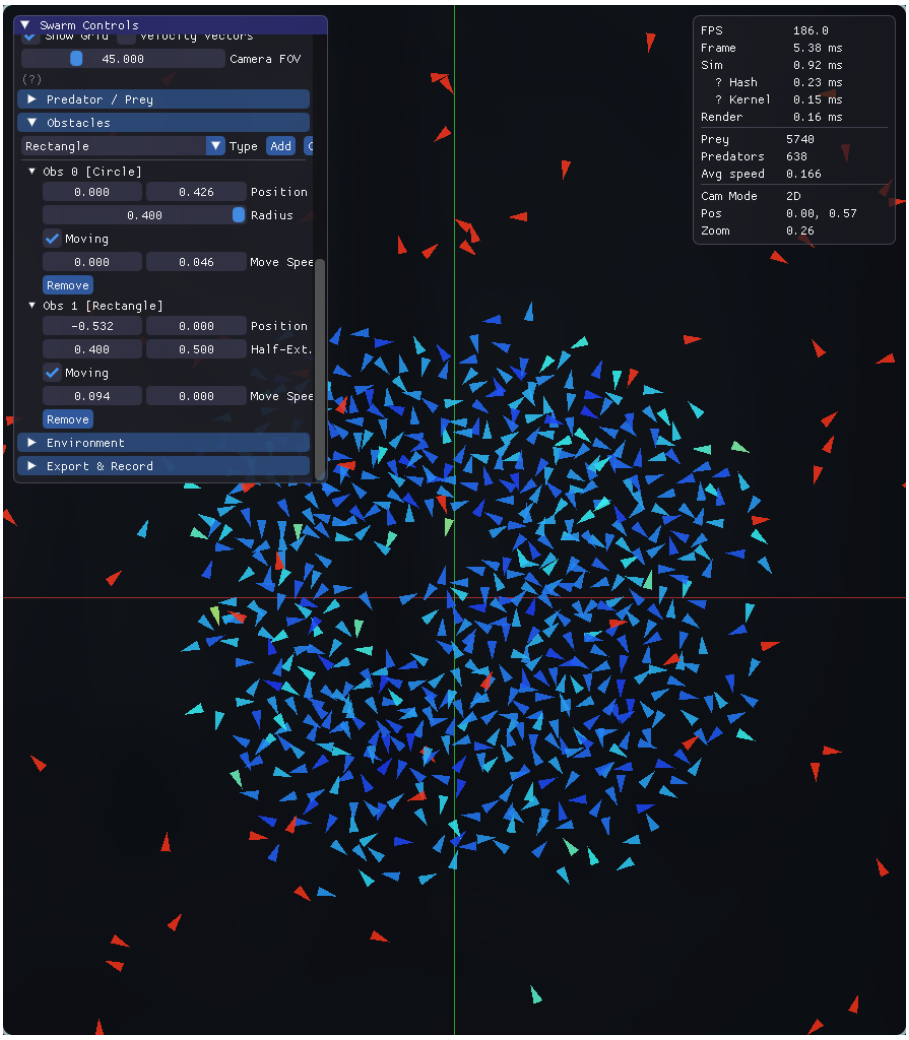


Figure 3.2: Swarm navigating around solid obstacles. The GPU ray-casting math cleanly parts the flock without requiring complex collision physics.

- **Boundary Steering (Soft Margins):** Instead of rigid walls that cause agents to abruptly bounce, the simulation uses "soft margins." When an agent crosses a predefined margin threshold (e.g., $x > 0.9$ or $x < -0.9$), a persistent turning force is applied opposite to the boundary, smoothly steering the flock back into the center of the domain.

- **Kinematics (V_{max}, F_{max}, and Time Scale):** Each agent is bound by a maximum speed limit V_{max} and a steering force limit F_{max}. Furthermore, `PREDATOR` agents utilize a distinct speed multiplier, allowing them to outpace prey during pursuit. Finally, a global Time Scale parameter (Δt_{scale}) multiplies the Δt during integration, allowing the simulation to smoothly operate in slow motion or fast-forward without affecting the physics stability.

3.2.4 Integration Scheme

At the conclusion of a simulation step, the cumulative acceleration is clamped to a global F_{max} to prevent infinite acceleration singularities. The physical state is then integrated using Semi-Implicit Euler integration:

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \mathbf{a}_t \cdot \Delta t \tag{3.7}$$

$$|\mathbf{v}_{t+\Delta t}| = \min(|\mathbf{v}_{t+\Delta t}|, V_{max}) \tag{3.8}$$

$$\mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v}_{t+\Delta t} \cdot \Delta t \tag{3.9}$$

Chapter 4

CUDA Architecture and GPGPU Implementation

4.1 CUDA Thread Hierarchy and Data Mapping

The simulation harnesses the NVIDIA Compute Unified Device Architecture (CUDA). The GPU is divided into multiple Streaming Multiprocessors (SMs), each capable of executing hundreds of threads concurrently via a SIMT (Single Instruction, Multiple Thread) architecture.

In this simulator, a 1:1 mapping exists between a simulated Agent and a CUDA thread. The kernel execution grid is dynamically sized based on the user-selected agent count:

```
1 int blockSize = 256;
2 int gridSize = (count + blockSize - 1) / blockSize;
3 boidsKernel<<<gridSize, blockSize>>>(...);
```

Listing 4.1: Kernel Execution Configuration

A block size of 256 was empirically determined to provide the optimal balance between register usage and warp occupancy, ensuring that SMs are fully saturated without causing register spilling to local memory.

4.2 Memory Optimization and Coalescing

Memory access latency is the primary bottleneck in any GPGPU application. To achieve maximum memory bandwidth, memory accesses must be *coalesced*. A coalesced access occurs when consecutive threads in a warp access consecutive memory addresses.

Because agents physically near each other in the 2D plane interact the most, the spatial hashing algorithm explicitly sorts the agent data array in GPU Global Memory every frame. This ensures that when a warp of threads loops through neighbor agents within a spatial grid cell, they are reading from contiguous, cache-aligned memory blocks.

4.3 Warp Divergence Mitigation

CUDA threads execute in warps (groups of 32). If threads within a warp take different execution paths in an `if-else` statement, the warp is forced to execute both paths sequentially, neutralizing the benefits of parallelism (warp divergence).

The neighbor querying logic was strictly optimized to avoid early-exit divergence. Rather than threads breaking out of loops at varying times, the neighborhood traversal operates on a strictly deterministic grid cell boundary structure. Conditional assignments are handled using ternary operators or arithmetic masking wherever possible to coerce the compiler into utilizing conditional move instructions (e.g., `PTX selp`) rather than branch instructions.

Chapter 5

Spatial Hashing Algorithm

5.1 The Uniform Grid Partitioning

To overcome the $O(N^2)$ algorithm cost, the simulation space is mathematically divided into a uniform grid. The dimensions of each cell are intrinsically linked to the perception radius R_{per}. Ideally, the cell size is exactly equal to R_{per}. This guarantees that any agent located within a specific cell only needs to search for neighbors within its own resident cell and the 8 directly adjacent surrounding cells.

5.2 Hash Function and Key Generation

At the start of every frame, an `assignCellsKernel` is launched. It calculates the 2D integer coordinates (cx, cy) of the agent based on its floating-point world position:

$$cx = \lfloor \frac{x_i}{CellSize} \rfloor \tag{5.1}$$

$$cy = \lfloor \frac{y_i}{CellSize} \rfloor \tag{5.2}$$

These 2D coordinates are subsequently compressed into a 1D scalar hash key using a large-prime mixing function to minimize modulo collisions:

```
1 __device__ unsigned int getHash(int cx, int cy, int tableSize
     ) {
2     return (unsigned int)((cx * 1610612741) ^ (cy *
         805306457)) % tableSize;
3 }
```

Listing 5.1: Spatial Hash Function

5.3 Parallel Sorting with Thrust

With each agent assigned a spatial hash key, the entire array of agent indices is sorted based on these keys using the `thrust::sort_by_key` algorithm. Thrust, included in the CUDA toolkit, provides a highly optimized parallel radix sort that executes entirely on the GPU device.

The sorting phase guarantees that agents residing in the same grid cell are packed adjacent to one another in the sorted index buffer.

5.4 Boundary Identification

Following the sort, a `findBoundariesKernel` is launched. It analyzes the sorted array of hash keys and detects boundaries where the key changes. It populates two arrays, `cell_start` and `cell_end`, indexing directly into the sorted agent list.

During the physics integration phase, a thread determining its neighbors merely calculates the hash of an adjacent cell, looks up the start and end indices from the boundary arrays, and loops precisely over that memory range. This reduces the search space from N agents to approximately K agents, achieving $O(1)$ spatial queries.

Chapter 6

Graphics Pipeline and CUDA-OpenGL Interoperability

6.1 The Device-to-Host Bottleneck

In a traditional computing architecture, a physics simulation runs on the CPU, and the calculated transform matrices are uploaded over the PCI-Express bus to the GPU for rendering. In a CUDA-accelerated simulation, the physics data already resides on the GPU. Transferring 200,000 position and velocity vectors from GPU VRAM back to System RAM (Device-to-Host), only to immediately send it back to the GPU for OpenGL rendering (Host-to-Device), introduces catastrophic latency.

6.2 Zero-Copy Interoperability

This simulator utilizes the `cudaGraphicsGLRegisterBuffer` API to establish a shared memory mapping between the CUDA compute context and the OpenGL graphics context.

1. During initialization, an OpenGL Vertex Buffer Object (VBO) is generated and its memory is pre-allocated.
2. This VBO is registered with CUDA, obtaining a `cudaGraphicsResource` pointer.
3. Every frame, the OpenGL resource is "mapped" to CUDA, yielding a raw float pointer.
4. The `boidsKernel` writes the integrated positions and velocities directly into this mapped pointer.

5. The resource is "unmapped", instantly returning ownership to OpenGL.

This zero-copy pipeline ensures that the high-frequency physics data never crosses the PCI-E bus, allowing the simulation to scale strictly linearly with GPU memory bandwidth.

6.3 Instanced Rendering

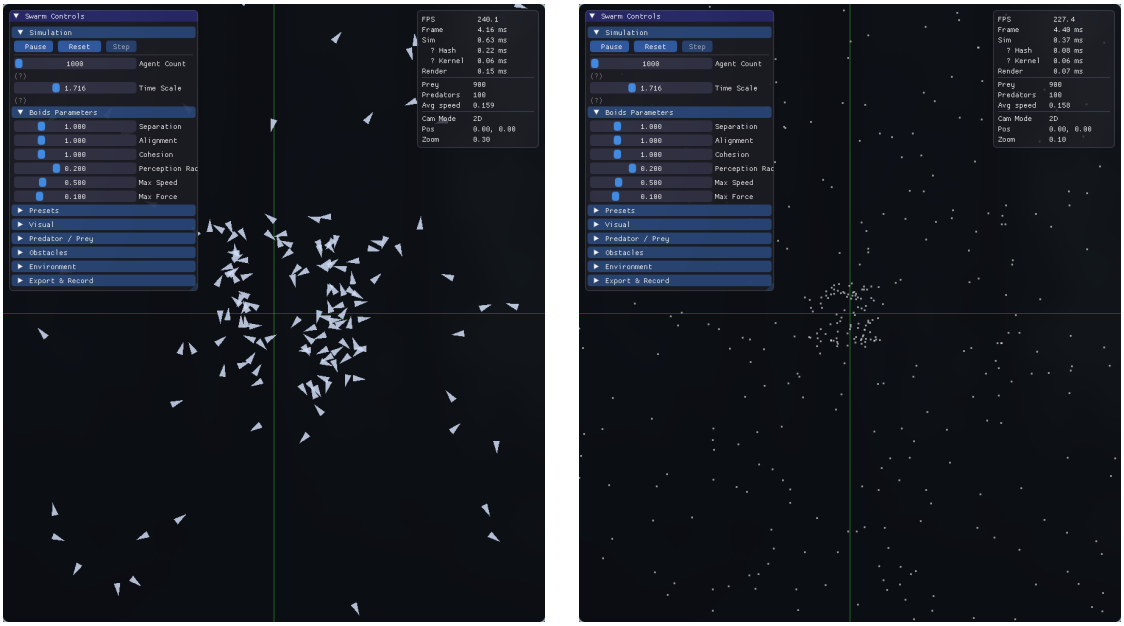
To render up to 200,000 distinct geometries, standard immediate mode or batched rendering is insufficient due to immense CPU draw-call overhead. To circumvent this, the simulation relies heavily on the `glDrawArraysInstanced` function. A single base geometry (a triangle representing a boid) is uploaded to the GPU once. A secondary VBO containing the shared instance data (the positions generated by CUDA) is then bound. A single OpenGL API call commands the GPU hardware to duplicate the base geometry 200,000 times, feeding a different position vector into the Vertex Shader for each individual instance.

6.4 Dynamic Level of Detail (LOD) and Overdraw

A critical technical challenge encountered during development was the "White Screen Crash"—a catastrophic drop to 2 FPS when zooming into dense agent clusters. Profiling revealed this was not a memory limitation, but an extreme fragment shader fill-rate bottleneck. Drawing hundreds of thousands of overlapping triangles resulted in billions of fragment rasterizations (overdraw).

To solve this, the renderer employs a dynamic LOD system tied to the camera's orthographic projection matrix.

- **Zoomed In ($Scale < 0.2$):** Agents are rendered as full geometric triangles with directional headings.
- **Zoomed Out ($Scale \geq 0.2$):** The pipeline switches from rendering triangles to `GL_POINTS`. This replaces expensive rasterization with single-pixel plotting, entirely circumventing the fill-rate bottleneck at macro scales and restoring FPS to 144+.



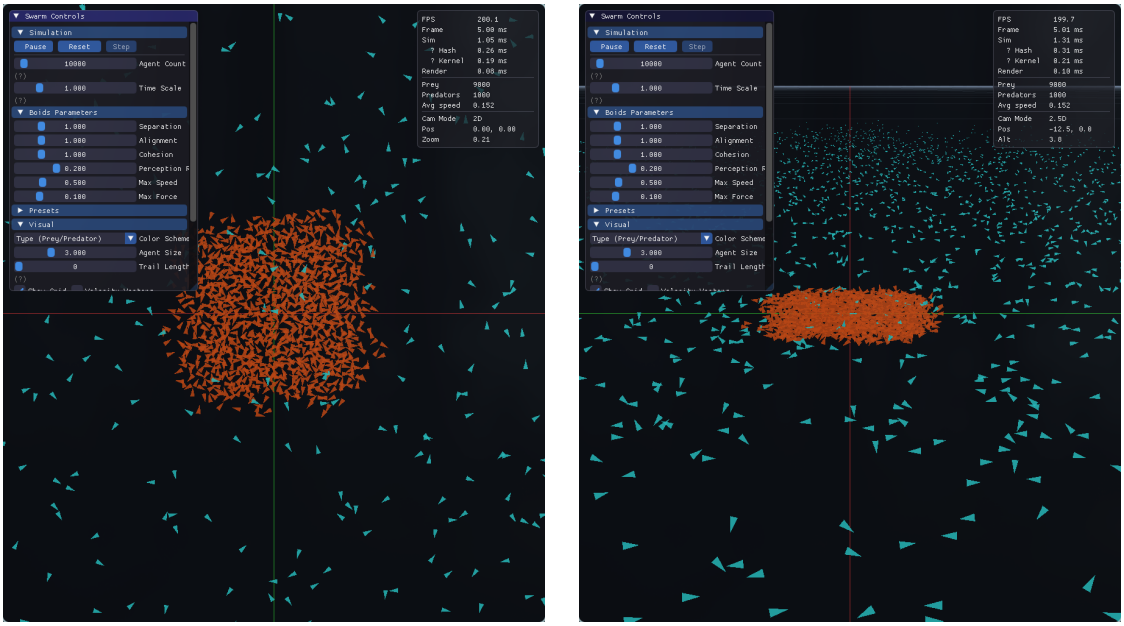
Zoomed In (Triangles) Zoomed Out (`GL_POINTS`)

Figure 6.1: Dynamic Level of Detail (LOD) in action. The engine seamlessly transitions from full instanced geometry to point-rendering to prevent fragment shader overdraw at macro scales.

6.5 Camera Architecture and Real-Time Controls

To provide varying degrees of spatial comprehension, the rendering engine supports dual camera projection matrices, actively navigable via a responsive keyboard control scheme:

- **2D Orthographic Projection:** The default viewing mode, projecting the X and Y coordinates onto a strictly flat, distance-agnostic plane. This view is ideal for mathematically analyzing the distribution and density of the swarm without perspective distortion.
- **2.5D Perspective Projection:** By introducing a field of view (FOV) and applying a perspective projection matrix, the camera simulates depth and parallax over the 2D plane. This creates a deeply cinematic visual experience, particularly when viewing the swarm from an acute grazing angle.
- **Navigation and Toggles (`WASD`, `QE`, `C`):** The camera operates as a free-flying observer. Traditional `WASD` keystrokes pan the camera across the mathematical plane, while `Q` and `E` dynamically alter the camera's Z-axis altitude (zoom/elevation). The `C` key serves as a rapid projection toggle, allowing users to seamlessly instantly swap the view frustum between 2D Orthographic and 2.5D Perspective modes during live simulation.



2D Orthographic 2.5D Perspective

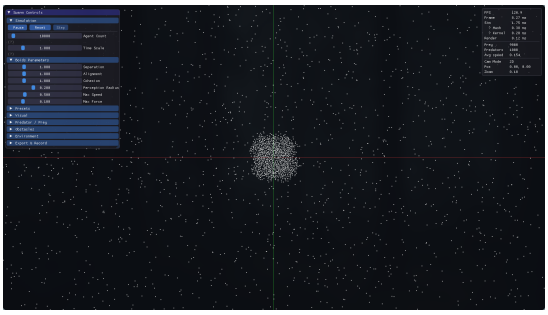
Figure 6.2: Comparison of the mathematical 2D Orthographic flat-plane projection against the cinematic depth achieved via the 2.5D Perspective projection.

6.6 Multi-Shader Pipeline

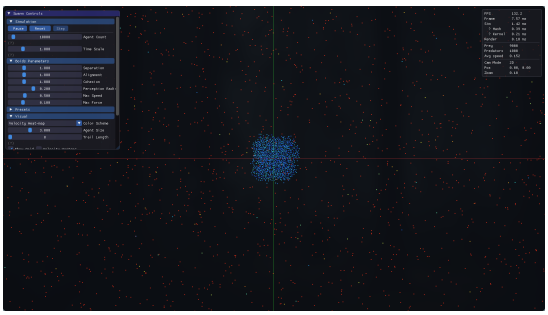
The visual fidelity of the simulation relies on a bifurcated shader architecture, utilizing distinct vertex and fragment programs to represent different data domains:

- **Agent Shader Pipeline (`agent.vert/frag`):** Dedicated to the primary geometry of the boids. It utilizes the instanced velocity data passed from CUDA to dynamically compute rotation matrices in the Vertex Shader, ensuring each triangle points precisely along its heading vector. The Fragment Shader maps speed (magnitude of velocity) to a color gradient, visually representing the kinetic energy of the flock.

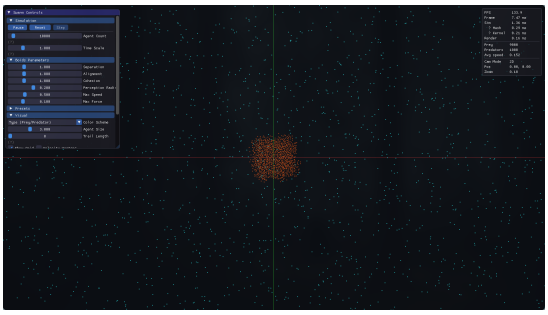
- **Trail Shader Pipeline (`trail.vert/frag`):** To visualize the historical flow of the swarm, a secondary trail pipeline renders line segments. This requires maintaining a historical ring buffer of positions. The Trail Fragment Shader applies distance-based alpha decay, creating a seamless fading effect that illustrates the aerodynamic wake of the swarm without requiring expensive particle emission systems.



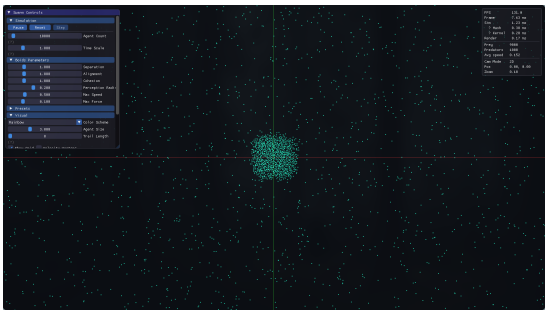
Uniform Mode



Velocity Heat-map



Prey/Predator Type



Rainbow Mode

Figure 6.3: Comparison of the four distinct shader visualization modes supported by the rendering pipeline.

Chapter 7

Performance Evaluation and Benchmarking

7.1 Hardware and Testing Methodology

To validate the scalability of the $O(N \cdot K)$ spatial hashing model against varying tiers of parallel architecture, the simulation was benchmarked across three distinct systems:

1. **Entry-Level / Mobile:** NVIDIA GeForce MX130
2. **Mid-Range Desktop:** NVIDIA GeForce RTX 3050
3. **Current-Gen Desktop:** NVIDIA GeForce RTX 4060

All tests were executed at a fixed 1080p resolution in 2D Orthographic mode with the *Velocity Heat-map* shader active.

7.2 Scalability Analysis (FPS vs. Agent Count)

The primary metric for real-time simulation is maintaining a framerate above the 60 Hz display threshold. As shown in Figure 7.1, the $O(N \cdot K)$ complexity allows even an entry-level MX130 to maintain > 60 FPS up to 70,000 agents. The RTX 4060 demonstrates massive throughput, rendering 100,000 agents at 470 FPS.

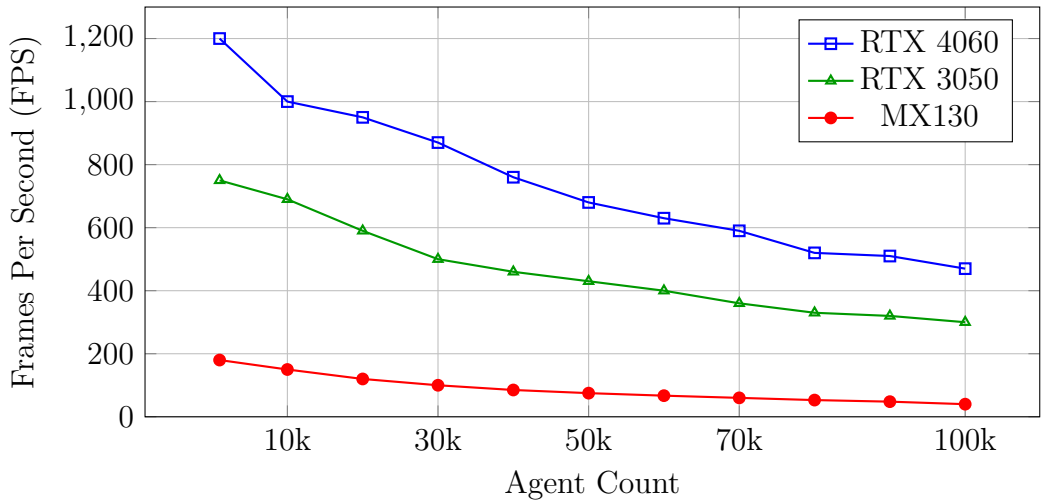


Figure 7.1: Hardware scalability comparison showing FPS decay as agent count increases from 1,000 to 100,000.

7.3 CUDA Execution Profiling and Bottleneck Identification

To capture empirical execution metrics without relying on external profiling software (like NVIDIA Nsight), the simulation integrates native hardware timers directly into the C++ simulation loop using the `cudaEventRecord` API. This ensures millisecond-accurate timing of the GPU kernels without stalling the CPU thread. These live diagnostic metrics—including the Thrust Radix Sort execution time and the Physics Integration Kernel time—are actively streamed back to the host and rendered in real-time on the Dear ImGui Performance Overlay.

While the overall framerate scales exceptionally well, this deep profiling uncovers the exact point of hardware saturation. The data visualized below was captured dynamically from the UI overlay on the MX130 to exaggerate the bottleneck conditions.

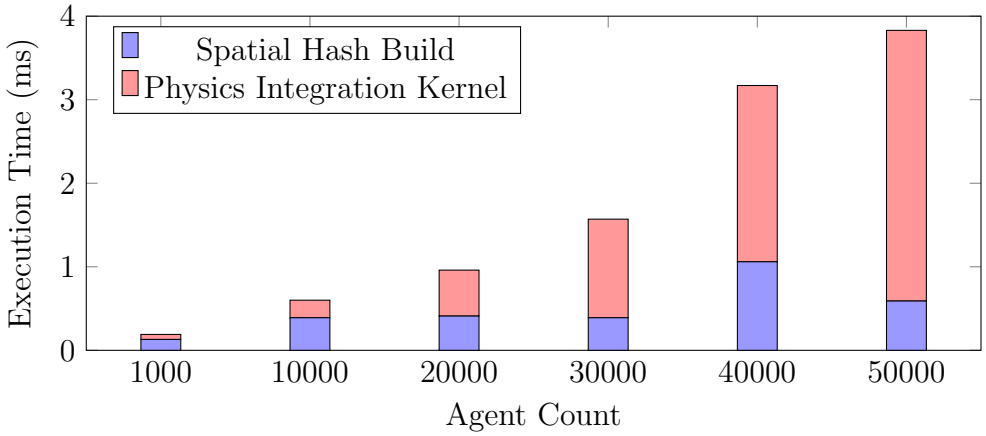


Figure 7.2: CUDA Pipeline breakdown on the MX130. The Physics Kernel execution time aggressively overtakes the Spatial Hashing time beyond 30,000 agents due to the exponential rise in grid cell occupancy queries.

As visualized in Figure 7.2, at lower counts (1k-10k), the overhead of Thrust Radix Sorting the Spatial Hash actually outweighs the Physics Kernel integration. However, as density increases towards 50,000 agents, grid cells become heavily congested. Consequently, the constant $O(K)$ neighbors queried per agent begins to grow substantially, resulting in the Physics Integration step ballooning to 3.24 ms and dominating the pipeline's execution time.

Chapter 8

User Interface and Interactive Ecosystem

8.1 Dear ImGui Integration

The application features a robust real-time diagnostic and control interface powered by Dear ImGui. Because the simulation operates asynchronously via the CUDA default stream, ImGui runs entirely on the CPU thread, submitting draw commands independently of the physics workload.

The interface is logically partitioned into functional tabs, providing absolute control over the simulation parameters:

- **Steering Weights:** Sliders to adjust the Separation, Alignment, and Cohesion forces in real-time.
- **Kinematics:** Tuning maximum speed, maximum force, and perception radii.
- **Environment:** Injecting global wind vectors, placing mouse-driven attractors, and spawning arbitrary obstacles dynamically.
- **Predator-Prey Ecosystem:** Adjustments for the *Predator Ratio* (which schedules a safe spatial re-initialization), *Predator Speed Multiplier*, and *Fear Weight*. Furthermore, the interface supports real-time manipulation of the ecosystem state; users can trigger `addAgent(PREDATOR)` or `convertRandomAgent()` to inject or mutate individual agents live, directly interfacing with the GPU device memory without halting the continuous CUDA physics stream.

8.2 Visual Tab and Rendering Overrides

Beyond physical parameters, the UI exposes a "Visuals" tab, directly interfacing with the `RenderOptions` struct sent to the OpenGL pipeline each frame:

- **Color Schemes:** Users can toggle between four distinct shading models:
  - *Uniform:* A solid, single color for all agents to maximize rendering performance.
  - *Velocity Heat-map:* A dynamic gradient mapping an agent's current speed (velocity magnitude) to color, providing immediate visual feedback on kinetic energy.
  - *Type (Prey/Predator):* A binary color mapping distinguishing between `PREY` and `PREDATOR` agent types, crucial for observing ecosystem dynamics.
  - *Rainbow:* An aesthetic spatial gradient that maps world-space coordinates or agent IDs to a full RGB spectrum.
- **Agent Geometry:** Sliders to manipulate global *Agent Size*, allowing for clearer visualization at macro scales.
- **Diagnostic Overlays:** Toggles to display individual *Velocity Vectors* as debug lines, or to *Show Grid* to visualize the underlying spatial hashing partitions.
- **Trail Dynamics:** Controls to adjust the maximum *Trail Length* and alpha decay rates, critical for maintaining high frame rates while still visualizing historical swarm trajectories.

8.3 State Management, Exporting, and Recording

To facilitate structured demonstration, reproducible research, and media generation, the UI implements a comprehensive serialization and output engine:

- **Scenario and Preset Serialization:** Users can construct specific parameter sets (e.g., "Tight Murmuration", "Predator Evasion") and append the `SimParams` struct to `config/presets.json` via the "Export Params" feature. A scenario orchestration system allows for scripted narrative demonstrations over a defined timeline.

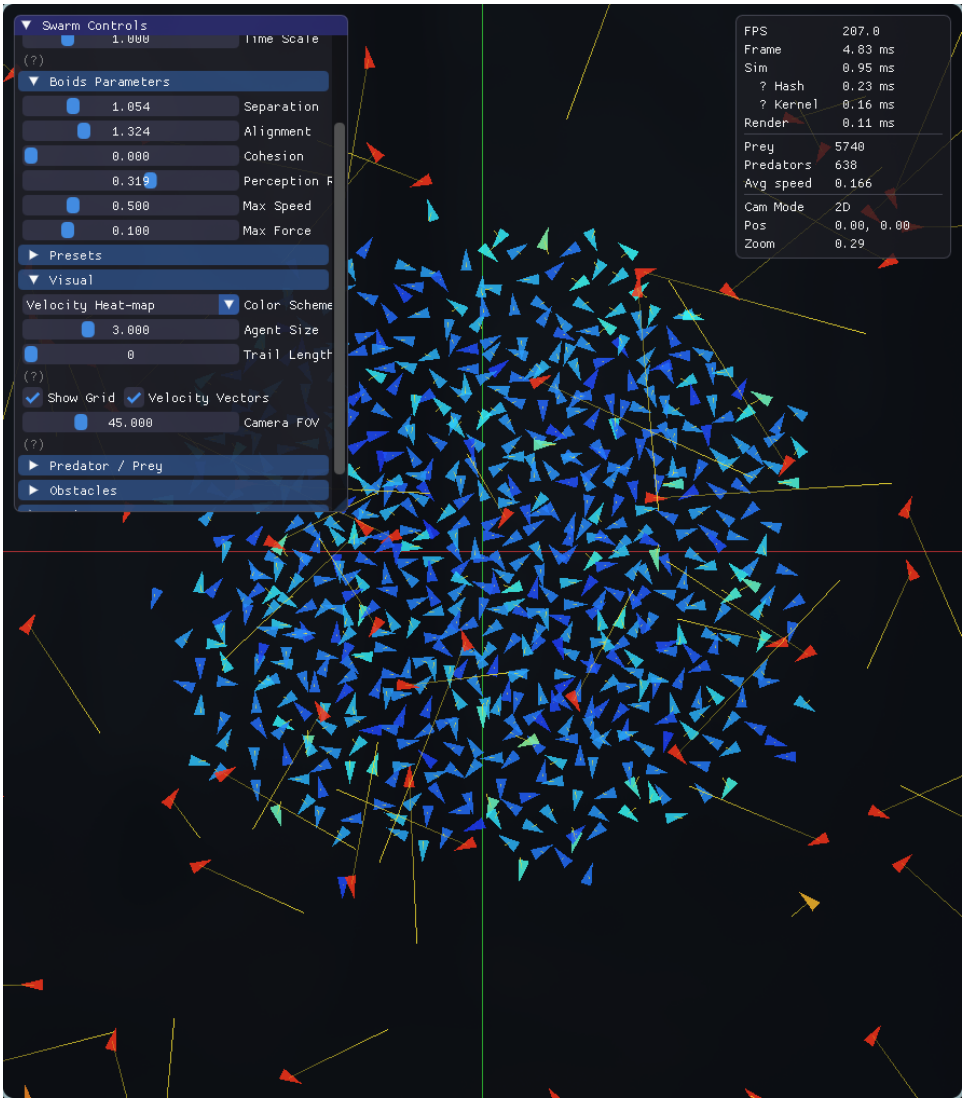


Figure 8.1: The real-time ImGui diagnostic overlay. Displaying the individual *Velocity Vectors* and the underlying *Spatial Hashing Grid* directly inside the visualization pipeline.

- **Full State Export/Load:** Beyond simple parameters, the "Export State" functionality serializes the entire simulation context—including `RenderOptions` and the exact placement and velocities of all dynamic obstacles—into a comprehensive JSON file. "Load State" fully reconstructs this ecosystem and triggers a re-initialization.

- **High-Fidelity Media Capture:** Researchers can capture the visual output asynchronously. "Screenshot" extracts the current OpenGL framebuffer to a PNG in the `screenshots/` directory. "Start Recording" initiates a sequential frame-dump to the `frames/` directory, allowing the generation of uncompressed 60FPS video using external tools like FFmpeg without tying the simulation logic to a video encoder's latency.

8.4 Scripted Scenarios and Orchestration

Beyond static presets, the engine features a temporal orchestration system (`scenarios.cpp`) capable of manipulating parameters over time to demonstrate complex emergent phenomena. Four distinct scenarios are natively supported:

1. **Murmuration:** Spawns 15,000 agents with high alignment and dynamic, oscillating wind vectors to simulate sweeping avian flight patterns.

2. **Predator Attack:** Spawns 10,000 agents with a 5% predator ratio. At exactly $T = 5.0s$, the "Fear Weight" parameter spikes globally to 8.0f, inducing a dramatic, synchronized radial scatter event.

3. **Obstacle Course:** Spawns 8,000 agents navigating a field of static circular obstacles. At $T = 10.0s$, the static obstacles suddenly acquire random velocity vectors, forcing the swarm to adapt to a dynamic, chaotic environment.

4. **Migration:** Spawns 12,000 agents reacting to a high-strength global attractor. The attractor's coordinates continuously shift across the simulation boundary, forcing the swarm to organize into trailing migratory columns.

Chapter 9

Future Work and Extensions

While the current architecture represents a highly optimized 2D simulation, several avenues remain for expansion:

9.1 Three-Dimensional Volumetric Simulation

The most direct extension is expanding the 2D Cartesian plane into a 3D volumetric space. This requires transitioning the spatial hash function from a 2-tuple (cx, cy) to a 3-tuple (cx, cy, cz), recalculating boundary interactions, and upgrading the rendering pipeline to handle depth buffering and projective 3D camera matrices.

9.2 Multi-GPU Peer-to-Peer Execution

For simulations exceeding 1 million agents, a single GPU's VRAM and streaming multiprocessors become insufficient. Utilizing NVIDIA's NVLink interconnect, the uniform spatial grid could be dynamically partitioned. By allocating the left hemisphere to GPU0 and the right to GPU1, only agents located near the spatial meridian would require cross-device peer-to-peer memory access, massively scaling the simulation ceiling.

9.3 Machine Learning Driven Behaviors

Currently, the weights for separation, alignment, and cohesion are statically defined by the user. By integrating a deep reinforcement learning framework (such as TensorRT), agents could actively mutate their weights using evolutionary algorithms. Fitness functions could be designed to reward minimal energy expenditure or max-

imum predator evasion duration, allowing the simulation to autonomously discover optimal biological flocking parameters.

Chapter 10

Conclusion

This project successfully demonstrates the immense computational superiority of GPGPU architectures in simulating complex, emergent biological systems. By translating the mathematically simplistic but computationally heavy $O(N^2)$ Boids algorithm into an optimized $O(N \cdot K)$ CUDA implementation, the historical limits of CPU-bound agent simulations have been shattered.

The synthesis of highly optimized data structures—namely uniform grid spatial hashing and parallel radix sorting—with low-level hardware interoperability (zero-copy VBO sharing) yields a system capable of modeling 200,000 distinct entities at interactive frame rates. The resulting real-time visualization not only serves as a mesmerizing display of emergent complexity but also stands as a robust educational framework demonstrating the practical application of parallel computing principles, memory coalescing, and graphics pipeline optimization.

References

[1] Reynolds, C. W. (1987). *Flocks, herds and schools: A distributed behavioral model*. ACM SIGGRAPH Computer Graphics, 21(4), 25-34.

[2] NVIDIA Corporation. (2023). *CUDA C++ Programming Guide*.

[3] Green, S. (2008). *Particle Simulation using CUDA*. NVIDIA Developer Technology.

[4] Khronos Group. (2021). *OpenGL Registry and Specifications*.

[5] Hoberock, J., & Bell, N. (2010). *Thrust: A Parallel Template Library*.

[6] Tu, X., & Terzopoulos, D. (1994). *Artificial fishes: Physics, locomotion, perception, behavior*. Proceedings of the 21st annual conference on Computer graphics and interactive techniques, 43-50.